

Relatório Técnico: Ferramenta Genérica de Diagnóstico (Shell Especialista)

Donvicton Monteiro, Yuck Arthur, Efraim Leite, Jadiel Henrique, John Wallex,
Neilton Gabriel

26 de junho de 2026

1 Arquitetura Implementada

A solução desenvolvida consiste em uma *shell* genérica de um Sistema Baseado em Conhecimento (SBC), construída na linguagem Python e estruturada com uma interface web utilizando o framework Streamlit. A arquitetura modular do agente foi dividida nos seguintes componentes principais:

- **Editor da Base de Conhecimento:** Módulo acessível via barra lateral da interface, que permite ao engenheiro do conhecimento ou especialista de domínio inserir, editar e excluir regras e hipóteses diretamente pela interface web, persistindo essas informações em um arquivo JSON local.
- **Base de Conhecimento:** Responsável por carregar, armazenar na memória (estado da sessão) e salvar fatos, regras e hipóteses.
- **Motor de Inferência:** O núcleo de raciocínio da aplicação, implementado de forma cíclica para interagir corretamente com o recarregamento de página do Streamlit.
- **Mecanismo de Explicação e Interação com o Usuário:** Integrado ao motor de inferência, o sistema provê respostas dinâmicas e captura as entradas do usuário por meio de botões visuais.

2 Estrutura de Representação do Conhecimento

O conhecimento do domínio é representado formalmente através de **fatos** e **regras de produção**.

- **Fatos:** Armazenados como pares de chave-valor em um dicionário (ex: {"pc_liga": "sim"}).
- **Regras:** Seguem estritamente o formato clássico de cláusulas de Horn (SE condição1 E condição2 ENTÃO conclusão). Elas são armazenadas como objetos da classe `Regra`, contendo um identificador único, uma lista de strings para as condições antecedente e uma string para o consequente.
- **Hipóteses:** Objetivos finais do sistema (diagnósticos), registrados em uma lista e avaliados em sequência pelo motor.

3 Estratégia de Inferência Utilizada

Para garantir robustez na dedução de dados lógicos, adotou-se a **Estratégia Híbrida (Encadeamento Misto)**.

1. **Forward Chaining (Encadeamento para Frente):** Ao iniciar um ciclo, o sistema varre a base deduzindo todos os fatos intermediários que podem ser inferidos passivamente, sem necessidade de perguntar ao usuário.
2. **Backward Chaining (Encadeamento para Trás):** Após esgotar as deduções diretas, o motor tenta provar as hipóteses da lista utilizando o raciocínio orientado por objetivos. Utiliza-se busca em profundidade (DFS), onde cada subcondição não provada torna-se um novo sub-objetivo. Se nenhuma regra resolve o sub-objetivo, o sistema interrompe a inferência, expõe a pergunta ao usuário e guarda o estado.

4 Implementação do Mecanismo de Explicação

O mecanismo de explicação é vital para garantir a transparência da IA.

- **"Por quê?"(Why):** Implementado no momento da interação. Quando o motor para a execução para perguntar algo, ele armazena o "objetivo atual" na memória. Se o usuário clicar em "Por quê?", a interface recupera esse objetivo e explica que a pergunta é um pré-requisito para validar a referida hipótese.
- **"Como?"(How):** Implementado via rastreamento contínuo. Durante o motor de inferência, cada vez que uma regra é ativada e deduz um fato novo, um dicionário (`como_explicacao`) mapeia o fato deduzido à regra específica que o gerou. Ao final, a interface exibe exatamente as variáveis que satisfizeram o antecedente.

5 Exemplos de Consultas Realizadas

- **Sistema:** O(a) `pc_liga` é/está 'sim' ou 'nao'?
- **Usuário:** Clica no botão Por quê?
- **Sistema:** "Porque estou avaliando a hipótese `diagnostico = falha_na_placa_mae` e saber sobre `pc_liga` é uma condição necessária."
- **Usuário:** Clica no botão sim.
- *(O sistema continua perguntando sobre 'video_na_tela', etc.)*
- **Sistema exibe:** DIAGNÓSTICO FINAL: `FALHA_NA_PLACA_MAE`.
- **Usuário:** Clica em "Como cheguei a essa conclusão?".
- **Sistema:** "Porque a regra **R6** foi ativada. Sabíamos que `suspeita = placa_mae_ou_video` E `bipes_na_inicializacao = nao`, ENTÃO concluímos que `diagnostico = falha_na_placa_mae`

6 Limitações e Possíveis Melhorias

Limitações:

- **Ausência de tratamento de incertezas:** As respostas são estritamente booleanas ("sim" ou "nao"), não suportando graus de certeza.
- **Conflito de Regras:** Caso a base de conhecimento seja preenchida com regras paradoxais, não há um tratador de conflito avançado no motor.
- **Múltiplas Conclusões:** A busca atualmente se encerra ao validar a primeira hipótese verdadeira da lista, não encontrando eventuais múltiplas doenças ou falhas simultâneas.

Possíveis Melhorias:

- Implementar Lógica Fuzzy ou Fatores de Certeza (CF) para suportar respostas do tipo "talvez" ou "muito provável".
- Integrar LLM (Generative AI) como sugerido pela especificação, utilizando a API da OpenAI ou Google Gemini para transformar as respostas estáticas da explicação ("Como" e "Por quê") em parágrafos de linguagem natural.
- Migrar a persistência JSON para um banco de dados relacional (ex: SQLite) ou NoSQL para suportar concorrência de múltiplos especialistas utilizando o editor simultaneamente.